Informed Search(Heuristic Search)

George Polya defines heuristic as:

"the study of the methods and rules of discovery and invention"

- This meaning can be traced to the term's Greek root, the verb *eurisco*, which means "I discover"

In AI, heuristics are formalized as

Rules for choosing those branches in a state space that are most likely to lead to an acceptable problem solution

When to use a Heuristic search?

- AI problem solvers employ heuristics in two basic situations:

1. A problem may not have an exact solution because of inherent ambiguities in the problem statement or available data (e.g.)

  - **Medical diagnosis:-** Doctors use heuristic to choose the most likely diagnosis based on patient's symptoms and description
  - **Computer Vision:-** Vision systems often use heuristics to select the most likely of several possible interpretations of a scene

2. A problem may have an exact solution, but the computational cost of finding it may be prohibitive

- A girl leaning against a women's shoulder while sitting on a bench.
- A mother with a child sitting on her lap and both are smiling.
- A woman and a child sit together and smile.
- A woman and a young girl sit close and pose for a photo.
- A woman and daughter smiling at the camera.

- A young girl wearing a blueish green shirt standing at the kitchen counter.
- A young girl with a bright bleu shirt eating food off of a countertop.
- A little girl in a bleu shirt is playing with something on a countertop.
- A small child is stirring food with a red spoon on a kitchen counter.
- A girl eats her food on the counter.

- A zebra grazing in a green pasture with cows.
- A zebra eating in a pasture with several horned animals in the background.
- A zebra is grazing on the green grass.
- A zebra feeding on green grass in the field with white animals in the distance.
- A zebra grazes on green grass while some other animals stand in the background.

Figure

Caption

Examples of image-description pairs of the three benchmark datasets. From the top to the bottom, they are come from the Flickr8K dataset, the Flickr30K dataset and the MS COCO dataset. Each image from these three datasets has five natural language sentences to describe the content of the image

When to use a Heuristic search?

- AI problem solvers employ heuristics in two basic situations:

1. A problem may not have an exact solution because of inherent ambiguities in the problem statement or available data (e.g.)

  - **Medical diagnosis**. Doctors use heuristic to choose the most likely diagnosis based on patient's symptoms and description
  - **Computer Vision**. Vision systems often use heuristics to select the most likely of several possible interpretations of a scene

2. A problem may have an exact solution, but the computational cost of finding it may be prohibitive

  - **Chess.** State space growth is combinatorially explosive, and brute force search techniques (BFS, DFS) may fail to find a solution within a reasonable amount of time

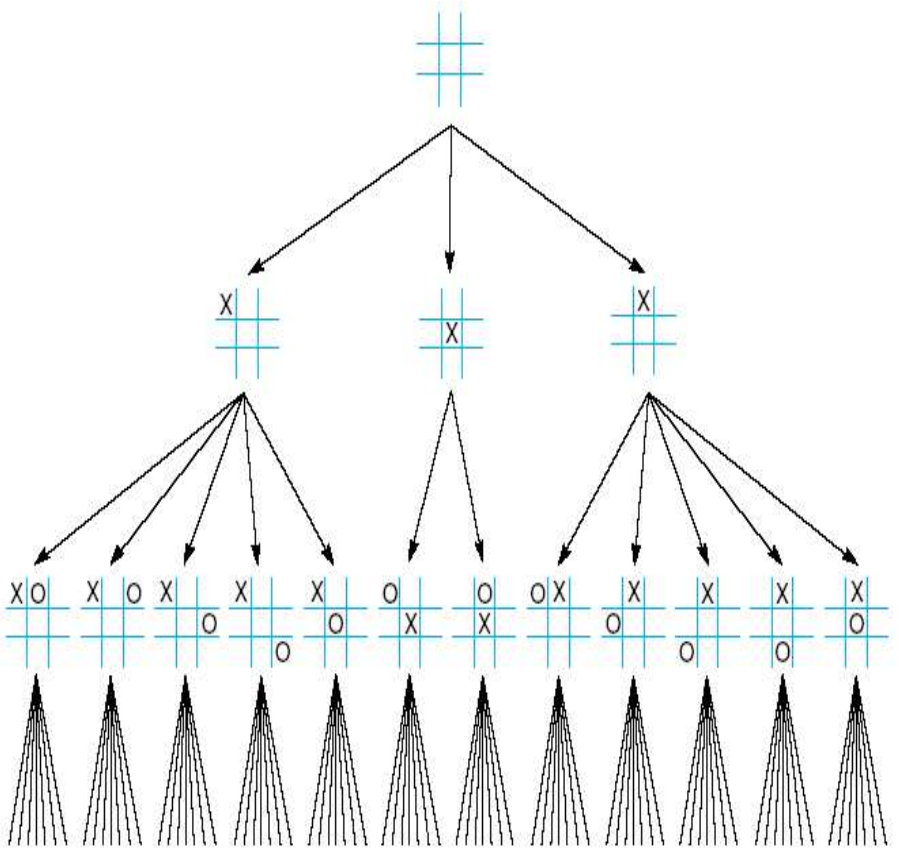
Does Heuristic guarantees a solution?

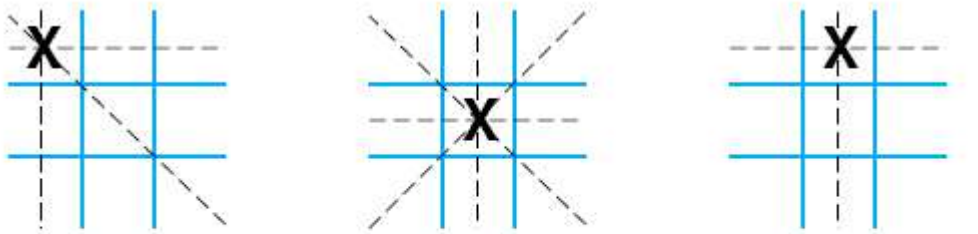
- Unfortunately, like all rules of discovery and invention, heuristic are fallible
- A heuristic is only an **informed guess** of the next step to take
- A heuristic can lead a search to a **suboptimal** solution or it may fail to find any solution at all, because heuristic use limited information of the present situation

Tic-Tac-Toe as an Example

- Consider heuristic in the game of tic-tac-toe
- A simple analysis gives the total number of paths as 9!
- **Symmetry** reduction decrease the search space
- Thus, there are not 9 but 3 initial moves:
  - ✓ to a corner
  - ✓ to the center of a side
  - ✓ to the center of the grid.

Further use of symmetry to the second level reduces the search space to 12X7!





Three wins through a corner square

Four wins through the center square

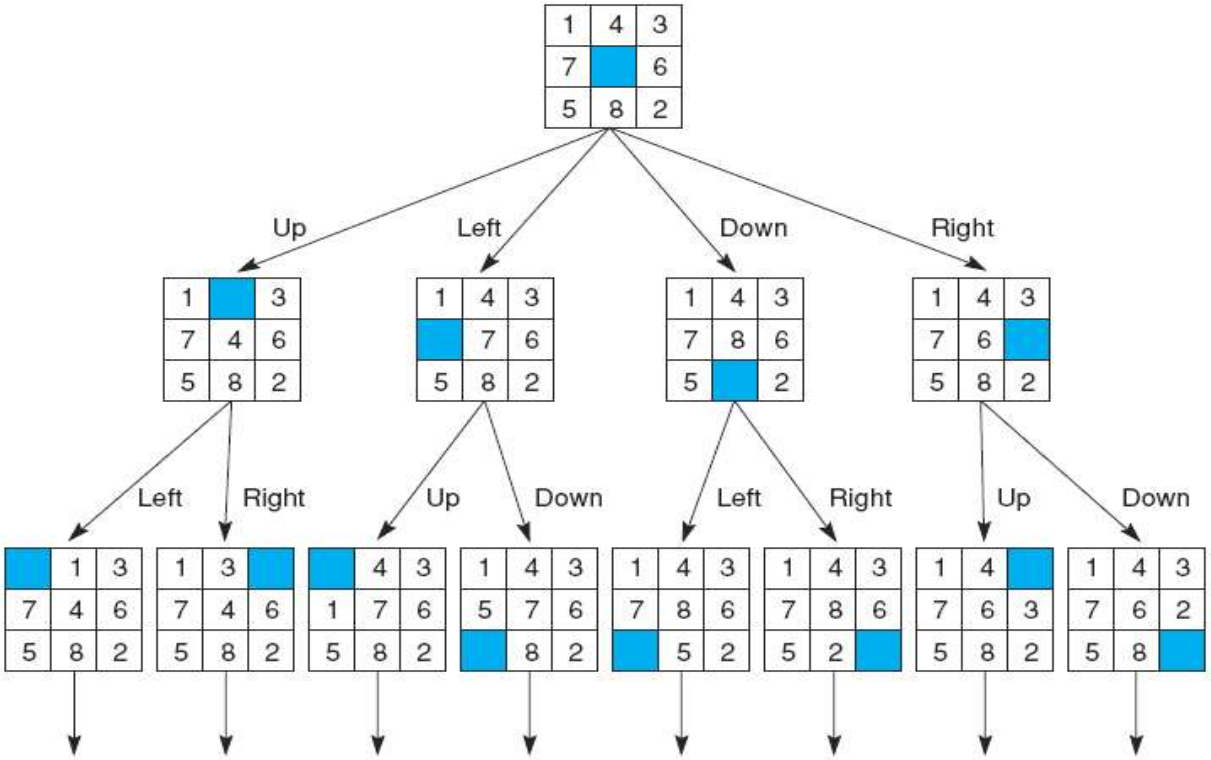
Two wins through a side square

A simple heuristic, can almost eliminate search entirely: we may move to the state in which X has **the most winning opportunity**

In this case, X takes the center of the grid as the first step

8-Puzzle as an Example

- Heuristics?



Local search and optimization

- **Local search= use single current state and move to neighboring states.**
- **Advantages:**
  - **Use very little memory**
  - **Find often reasonable solutions in large or infinite state spaces.**
- **Are also useful for pure optimization problems.**
  - **Find best state according to some *objective function*.**
  - **e.g. survival of the fittest as a metaphor for optimization.**

Simple Hill-Climbing

- The **simplest way** to implement heuristic search is through a procedure called hill-climbing.
- It expands the current state of the search and evaluate its children.
- The **Best** child is selected for further expansion.
- Neither its sibling nor its parent are retained
- Tic-Tac-Toe we just saw is an example.
- Since it keeps no history, the algorithm cannot recover from failures of its strategy
- A major problem of hill-climbing is their tendency to become stuck at **local maxima**

Hill Climbing Search

function HILL-CLIMBING(*problem*) **return** a state that is a local maximum

input: *problem*, a problem

local variables: *current*, **a node.**

neighbor, **a node.**

current ← MAKE-NODE(INITIAL-STATE[*problem*])

loop do

neighbor ← a highest valued successor of *current*

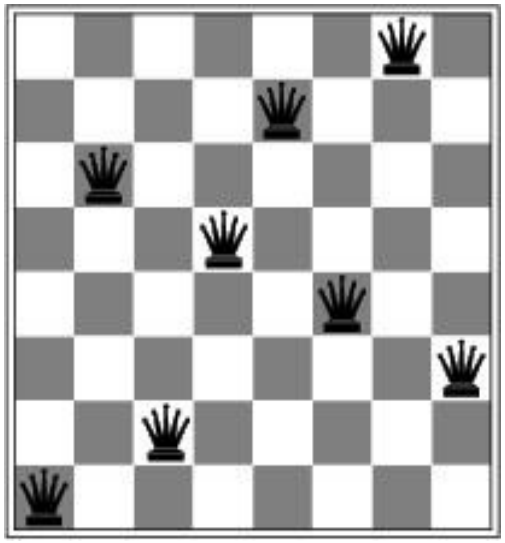
if VALUE [*neighbor*] ≤ VALUE[*current*] **then return** STATE[*current*]

current ← *neighbor*

Hill-climbing example
8-Queens Problem

a)

b)



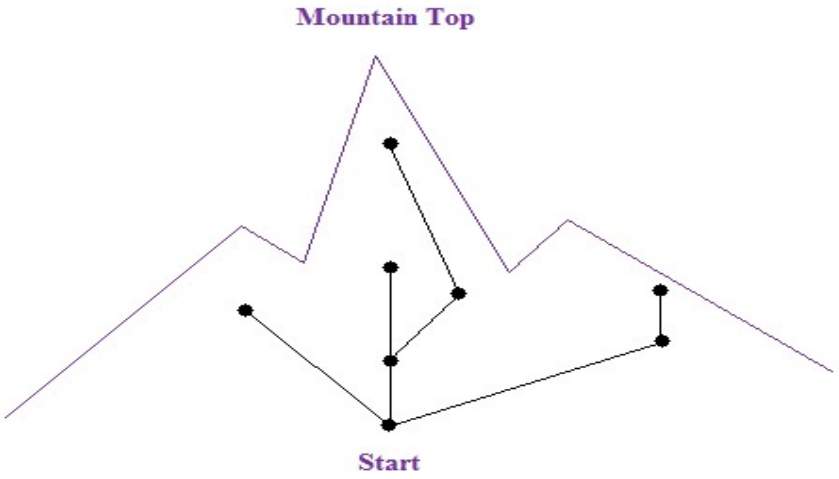
a) shows a state and the h-value for each possible successor.
b) A local minimum in the 8-queens state space (h=1).

Hill-climbing example

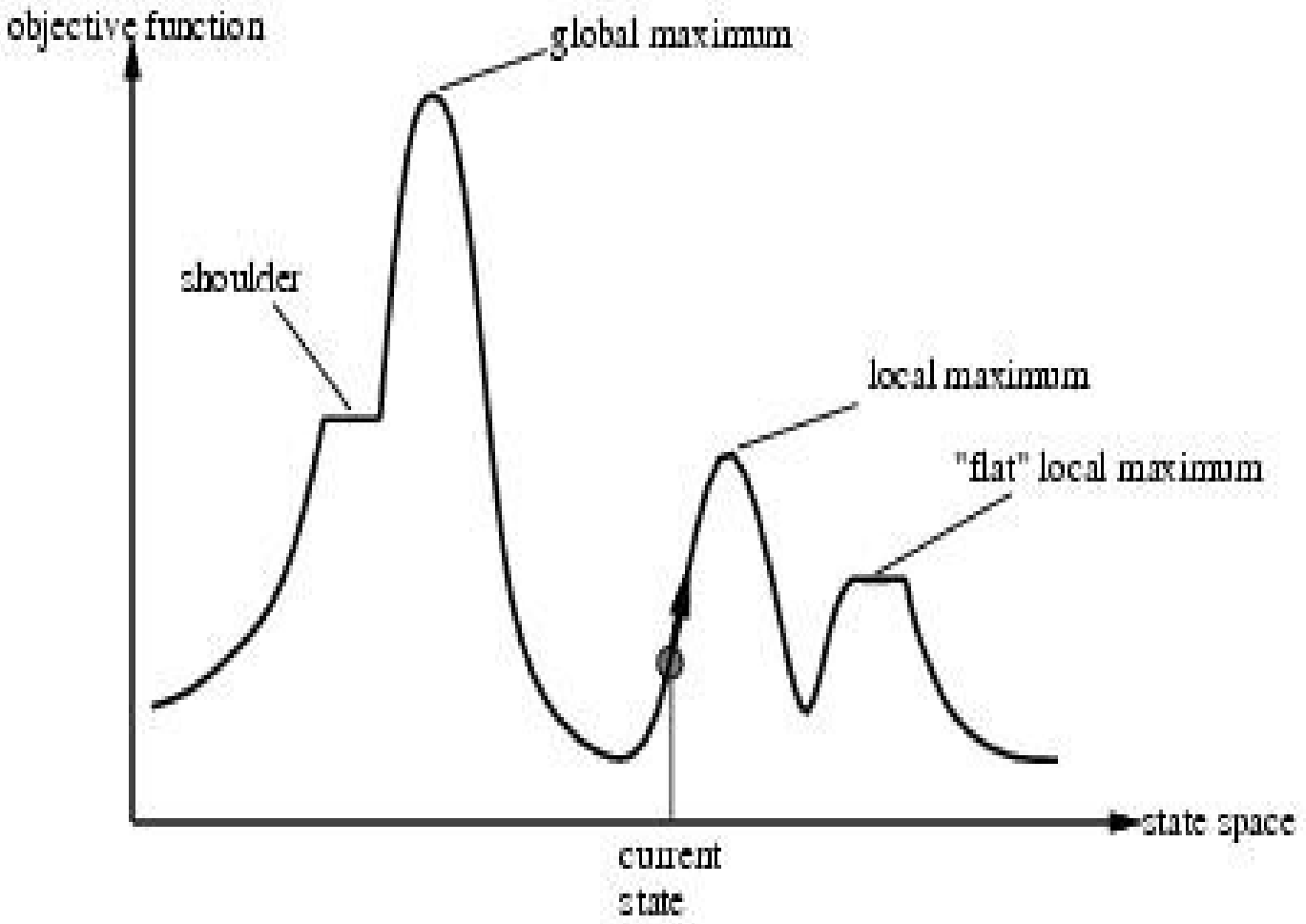
- 8-queens problem (complete-state formulation)
- Successor function: move a single queen to another square in the same column
- Heuristic function *h(n)*: the number of pairs of queens that are attacking each other (directly or indirectly)

Hill Climbing

- Hill Climbing is named for the strategy that might be used by an eager, but blind mountain climber:
- Go uphill along the closest-to-the-top path until you can go no further up

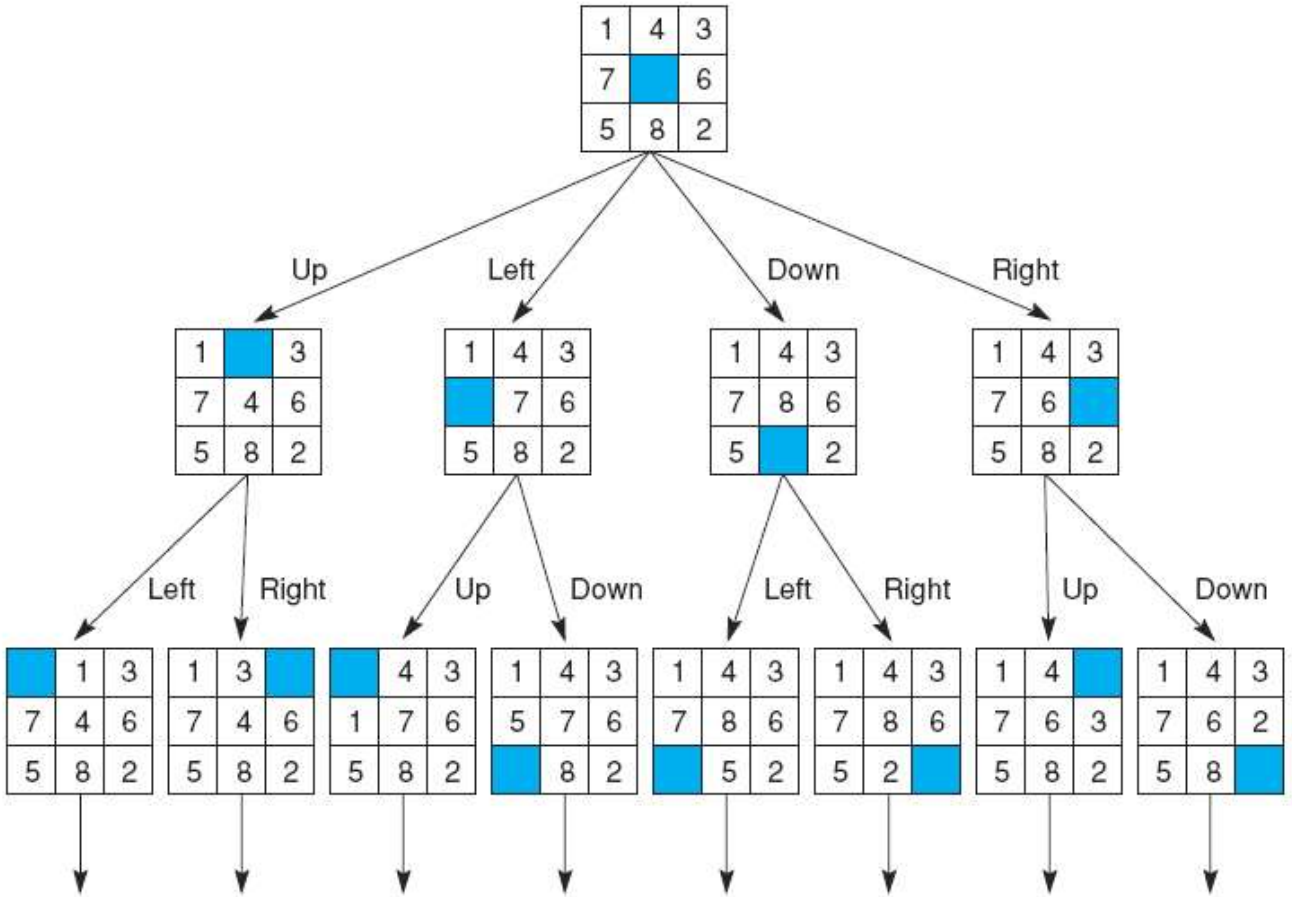


Local maxima



8-Puzzle- As an Example of local maxima in hill climbing

- An example of local maximum in games occurs in the 8-puzzle.
- Often, in order to move a particular tile to its destination, other tiles already in their position need to move out
- This is a necessary step but temporarily worsens the board state
- Thus, hill climbing is not useful to the 8-puzzle game


Up
Left
Down
Right
Left
Right
Up
Down
Left
Right
Up
Down

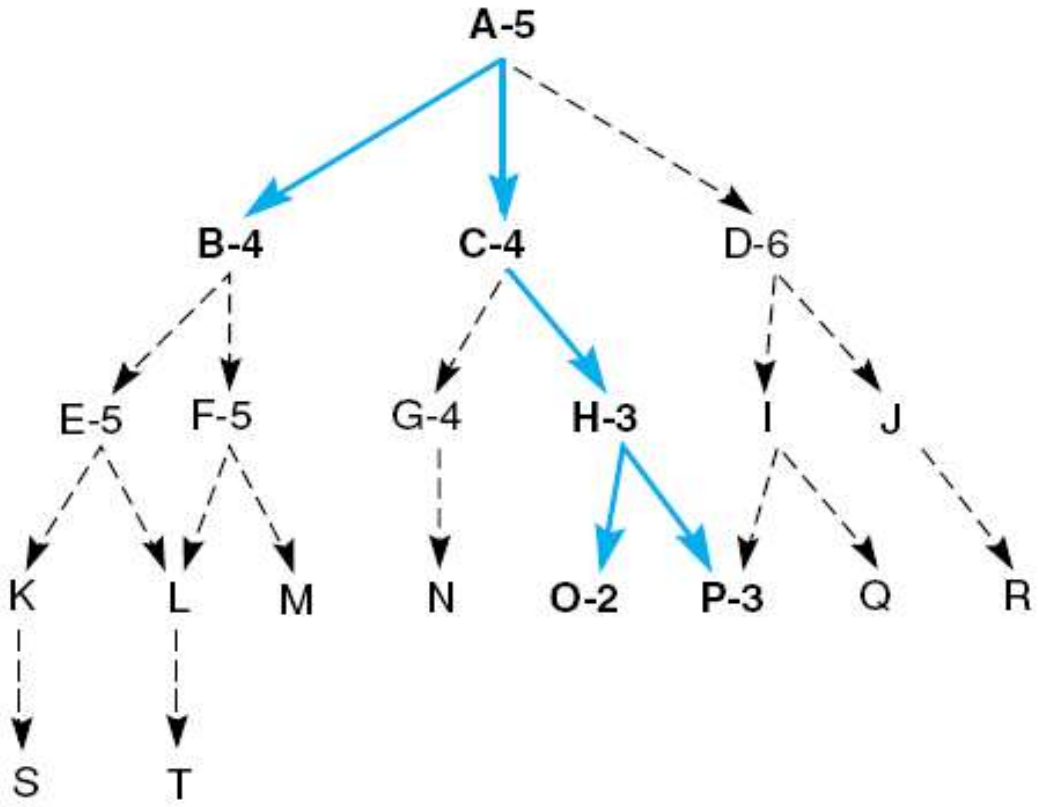
Best First Search

```
function best_first_search;

begin
   open := [Start];                                                    % initialize
   closed := [ ];
   while open ≠ [ ] do                                                 % states remain
      begin
         remove the leftmost state from open, call it X;
         if X = goal then return the path from Start to X
         else begin
                  generate children of X;
                  for each child of X do
                  case
                        the child is not on open or closed:
                           begin
                                 assign the child a heuristic value;
                                 add the child to open
                           end;
                        the child is already on open:
                           if the child was reached by a shorter path
                           then give the state on open the shorter path
                        the child is already on closed:
                           if the child was reached by a shorter path then
                              begin
                                 remove the state from closed;
                                 add the child to open
                              end;
                  end;                                                 % case
                  put X on closed;
                  re-order states on open by heuristic merit (best leftmost)
               end;
return FAIL                                                            % open is empty
end.
```

Best First Search

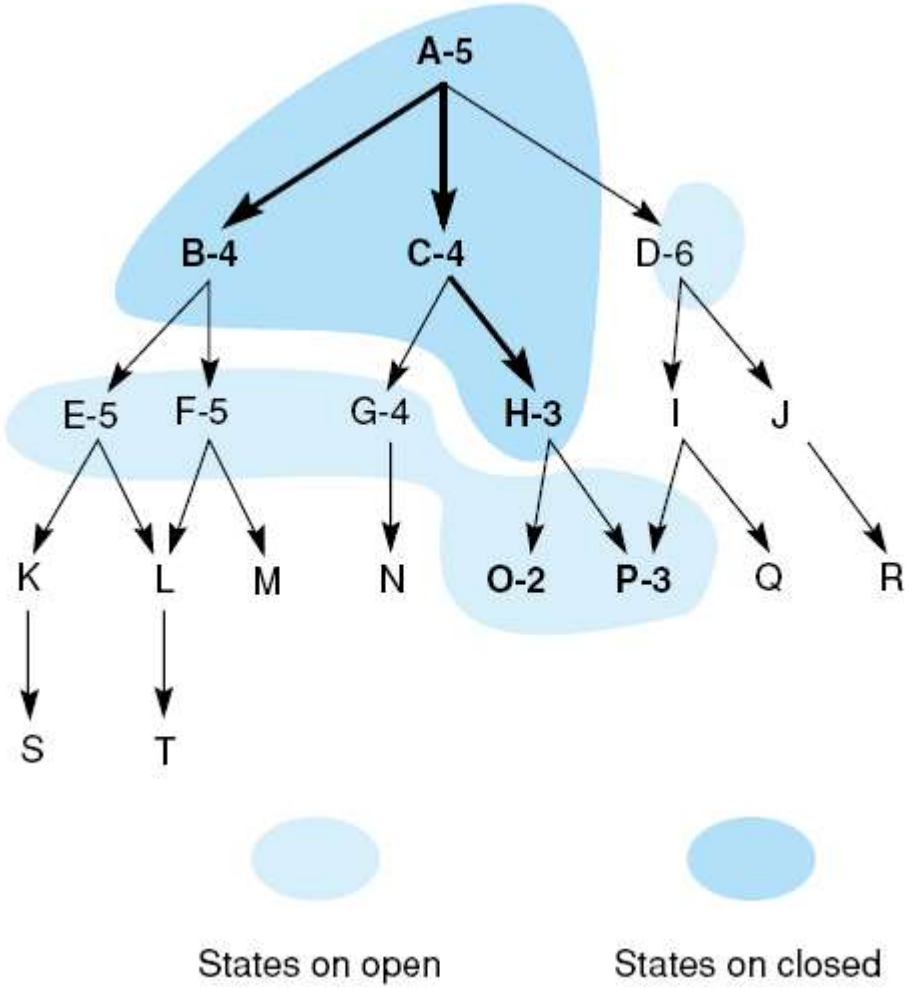
- Figure 4.4(Luger) Heuristic search of a hypothetical state space.



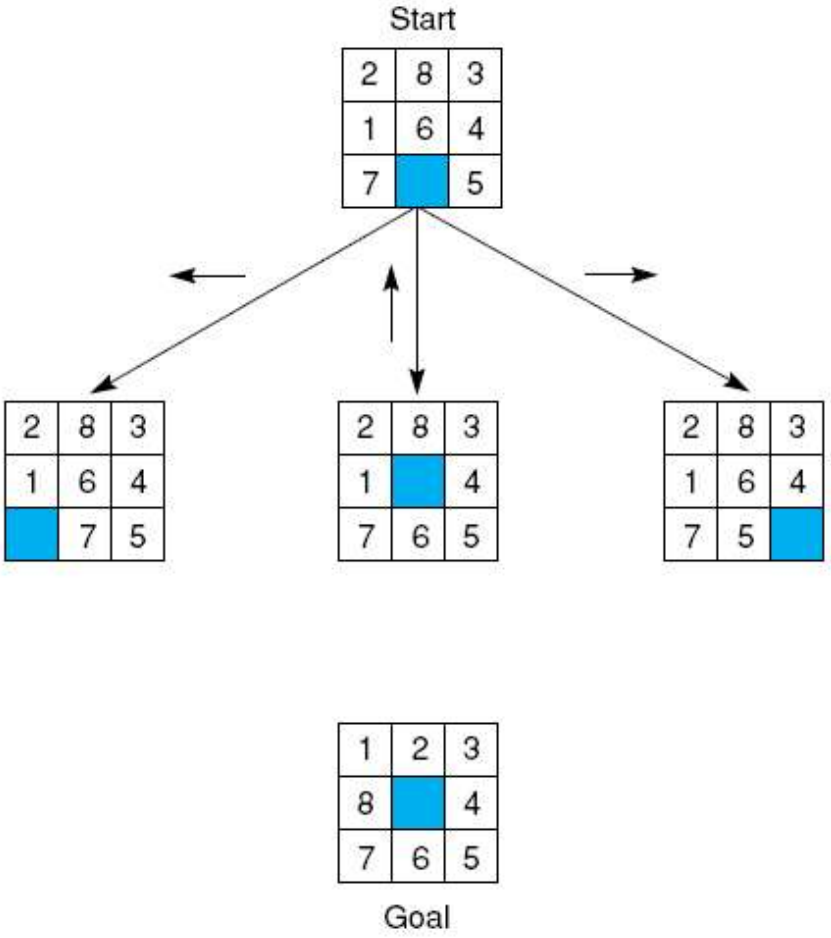
1. **open = [A5]; closed = [ ]**
2. **evaluate A5; open = [B4,C4,D6]; closed = [A5]**
3. **evaluate B4; open = [C4,E5,F5,D6]; closed = [B4,A5]**
4. **evaluate C4; open = [H3,G4,E5,F5,D6]; closed = [C4,B4,A5]**
5. **evaluate H3; open = [O2,P3,G4,E5,F5,D6]; closed = [H3,C4,B4,A5]**
6. **evaluate O2; open = [P3,G4,E5,F5,D6]; closed = [O2,H3,C4,B4,A5]**
7. **evaluate P3; the solution is found!**

A trace of the execution of best_first_search for Figure 4.4

Heuristic search of a hypothetical state space with open and closed states highlighted.



· Fig 4.12 The start state, first moves, and goal state for an example-8 puzzle.

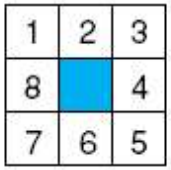


Heuristic for 8-Puzzle

- For the 8-puzzle game, we may add 3 different types of information into the code:
  - The simplest heuristic counts the tiles out of space in each state
  - A “better” heuristic would sum all the distances by which the tiles are out of space
- Both of these heuristics can be criticized for failing to acknowledge the difficulty of tile reversals
- That is, if two tiles are next to each other and the goal requires their being in opposite locations, it takes several moves to put them back

Best First Search

2 8 3 1 6 4 _ 7 5	**5**	**6**	**0**
2 8 3 1 _ 4 7 6 5	**3**	**4**	**0**
2 8 3 1 6 4 7 5 _	**5**	**6**	**0**
	Tiles out of place	Sum of distances out of place	2 x the number of direct tile reversals



Goal

Best First Search

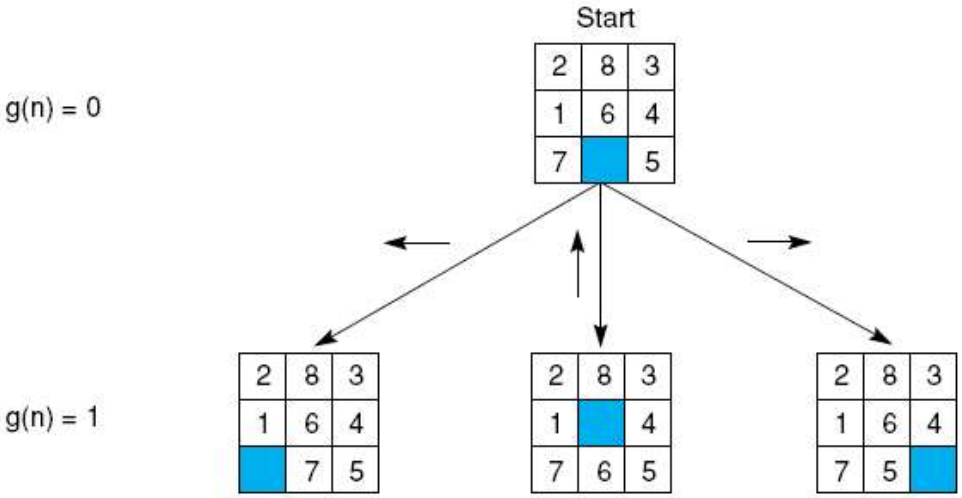
- In previous figure, the sum of distance heuristics does provide a much accurate estimate of the work
- This example illustrates the difficulty of devising good heuristics
- The design of good heuristic is an empirical problem; judgment and intuition help, but the final measure of a heuristic must be its actual performance
- If two states have the same or nearly the same heuristic evaluation, it is generally preferable to examine the state that is **nearest** to the root
- So that we may obtain the **shortest** path to the goal

Best First Search

- For 8-puzzle problem, let's make our evaluation function f to be :

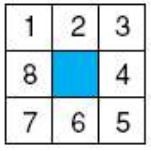
- f(n) = g(n) + h(n)
  - g(n) means the actual length of path from n to the root
  - h(n) is a heuristic estimate (sum of distance out of space) of the distance from state n to a goal

Best First Search

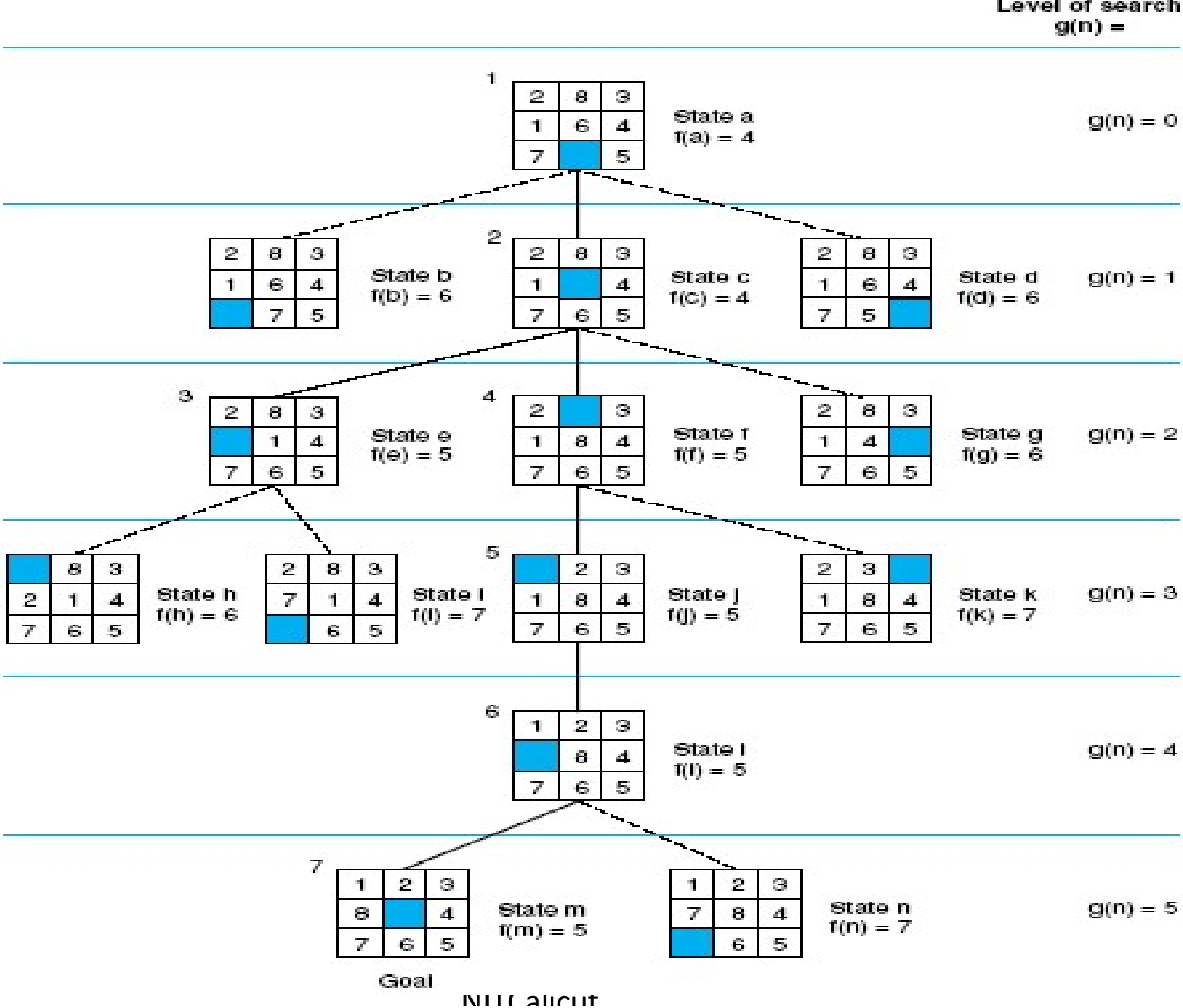


Values of f(n) for each state, **6** **4** **6**

where:
f(n) = g(n) + h(n),
g(n) = actual distance from n to the start state, and
h(n) = number of tiles out of place.



Goal


Level of search
g(n) =
1
State a
f(a) = 4
g(n) = 0
2
State b
f(b) = 6
State c
f(C) = 4
State d
f(d) = 6
g(n) = 1
3
State e
f(e) = 5
4
State f
f(f) = 5
State g
f(g) = 6
g(n) = 2
5
State h
f(h) = 6
State I
f(I) = 7
State J
f(j) = 5
State k
f(k) = 7
g(n) = 3
6
State I
f(I) = 5
g(n) = 4
7
State m
f(m) = 5
State n
f(n) = 7
g(n) = 5
Goal

Best First Search

- In step 3, both e and f have a heuristic of 5
- State e is examined first, producing h and i
- Compare with all children, select the best move

A* Search

- Best-known form of best-first search
- Idea: avoid expanding paths that are already expensive
- Evaluation function $f(n)=g(n) + h(n)$
  - $g(n)$ the cost (so far) to reach the node
  - $h(n)$ estimated cost to get from the node to the goal
  - $f(n)$ estimated total cost of path through n to goal

DEFINITION

ALGORITHM A, ADMISSIBILITY, ALGORITHM A*

Consider the evaluation function f(n) = g(n) + h(n), where

n is any state encountered in the search.

g(n) is the cost of n from the start state.

h(n) is the heuristic estimate of the cost of going from n to a goal.

If this evaluation function is used with the best_first_search algorithm of Section 4.1, the result is called *algorithm A*.

A search algorithm is *admissible* if, for any graph, it always terminates in the optimal solution path whenever a path from the start to a goal state exists.

If algorithm A is used with an evaluation function in which h(n) is less than or equal to the cost of the minimal path from n to the goal, the resulting search algorithm is called *algorithm* A* (pronounced "A STAR").

It is now possible to state a property of A* algorithms:

All A* algorithms are admissible.

A* Search

- A* search uses an admissible heuristic
  - A heuristic is admissible if it *never overestimates* the cost to reach the goal

Formally:

1. *h(n) <= h*(n)* where *h*(n)* is the true cost from *n*
2. *h(n) >= 0* so *h(G)=0* for any goal *G*.

e.g. *hSLD(n) (the straight line heuristic)* never overestimates the actual road distance

A*

1. **Put the start node s in OPEN.**

2. If OPEN is empty, exit with failure.

3. Remove from OPEN and place in CLOSED a node n for which f(n) is minimum.

4. If n is a goal node, exit with the solution obtained by tracing back

pointers from n to s.

5. Expand n, generating all of its successors. For each successor n' of n:

a. Compute g'(n'); compute f'(n')=g'(n')+h(n')

b. if n' is already on OPEN or CLOSED and g'(n')<g(n'), let g(n')=g'(n'), let

f(n')=f'(n'), redirect the pointer from n' to n and, if n' is on CLOSED, move it to OPEN.

c. if n' is neither on OPEN nor on CLOSED, let f(n')=f'(n'), attach a pointer from n' to n, and place n' on OPEN.

6. Go to 2.

DEFINITION

MONOTONICITY

A heuristic function h is monotone if

1. For all states n_i and n_j, where n_j is a descendant of n_i,

   $h(n_i) - h(n_j) \leq cost(n_i,n_j)$,

   where $cost(n_i,n_j)$ is the actual cost (in number of moves) of going from state n_i to n_j.

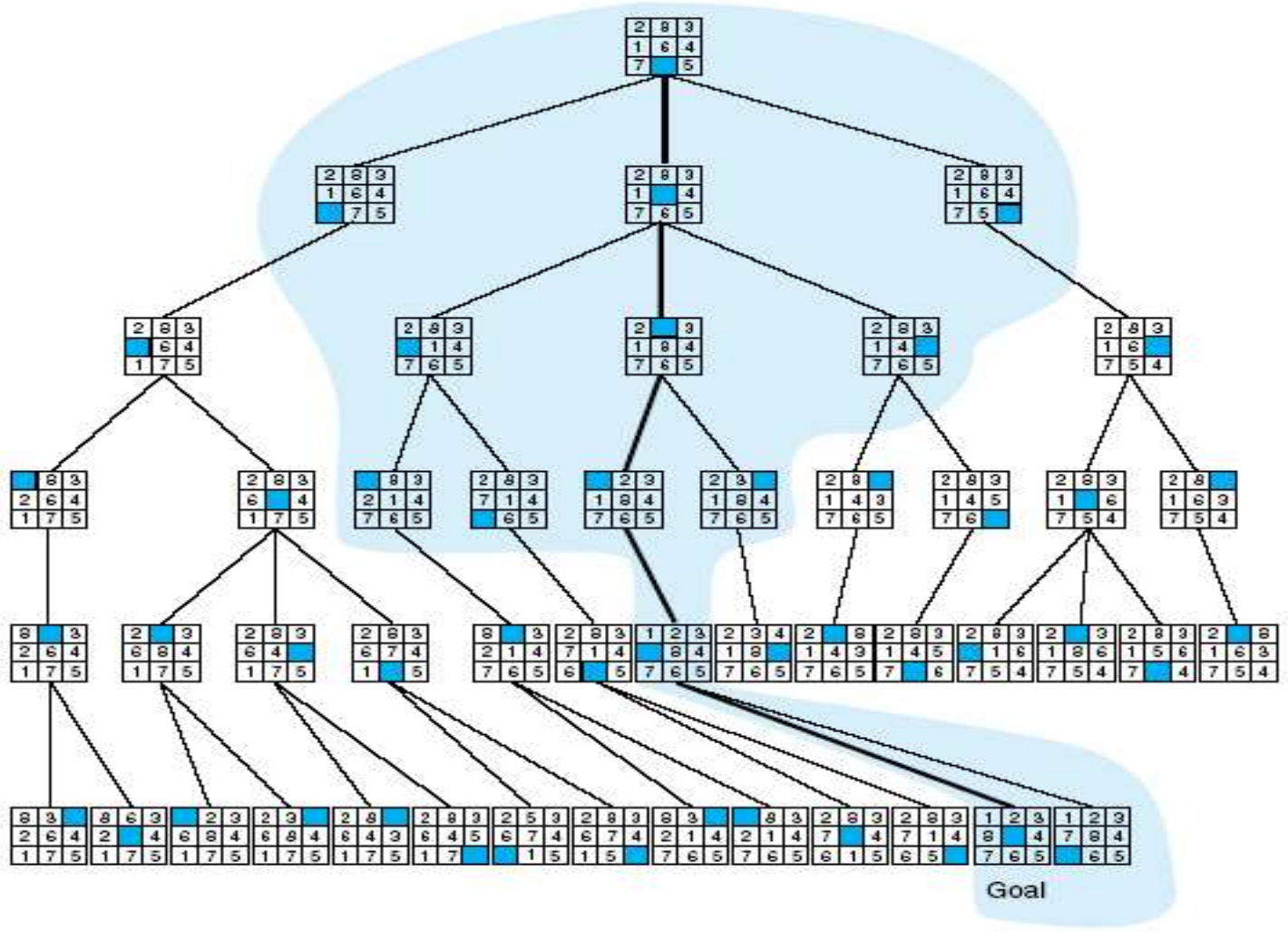
2. The heuristic evaluation of the goal state is zero, or h(Goal) = 0.

DEFINITION

INFORMEDNESS

For two A* heuristics h_1 and h_2, if $h_1(n) \leq h_2(n)$, for all states n in the search space, heuristic h_2 is said to be *more informed* than h_1.

Comparison of state space searched using heuristic search with space searched by breadth-first search. The proportion of the graph searched heuristically is shaded. The optimal search selection is in bold. Heuristic used is f(n) = g(n) + h(n) where h(n) is tiles out of place.



Admissible Heuristics and Search Algorithms

- **A heuristic *h(n)* is admissible if for every node *n*,**

  h(n)* ≤ *h*(n)*, where *h*(n)* is the true cost to reach the goal state from *n

- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic
- Example: *hSLD(n)* (never overestimates the actual road distance)
- A search algorithm is admissible if it returns an optimal solution path
  - [AIMA] calls such an algorithm optimal, instead of admissible
- Theorem: If *h(n)* is admissible, A* (as presented in this set of slides) is admissible on graphs

Consistent Heuristics

- A **heuristic is consistent if for every node *n*, every successor *n'* of *n* generated by any action *a*,**

  h(n) ≤ c(n,a,n') + h(n')

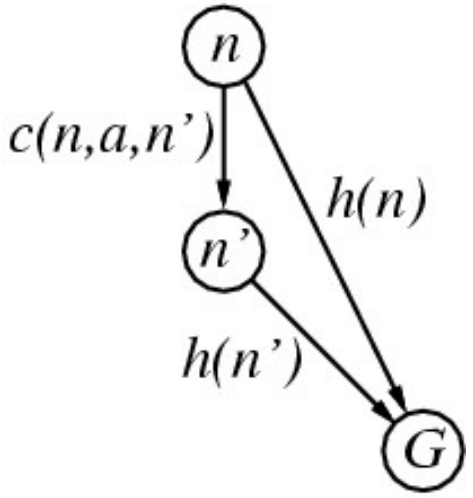
- If *h* is consistent, we have

  f(n') = g(n') + h(n')

  = g(n) + c(n,a,n') + h(n')

  ≥ g(n) + h(n)

  = f(n)



- i.e., *f(n)* is non-decreasing along any path
- Theorem: If *h(n)* is consistent, A* never expands a node more than once

Optimality of A* on Graphs

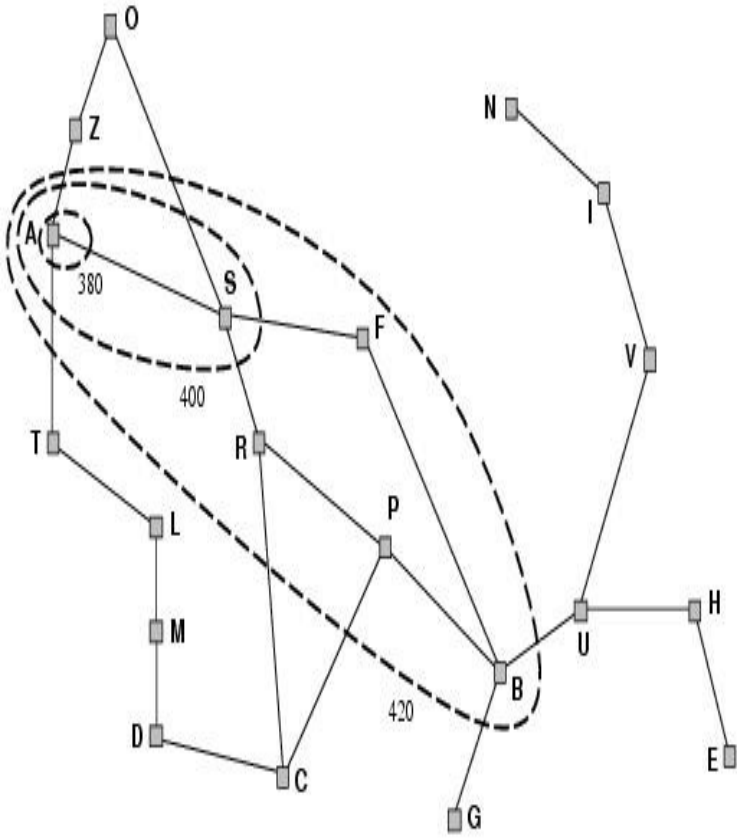
- A* expands nodes in order of increasing *f* value
- Contours can be drawn in state space
  - Uniform-cost search adds circles.
  - F-contours are gradually

Added:

1) nodes with *f(n)<C**

2) Some nodes on the goal

Contour (*f(n)=C**)



Contour I has all

Nodes with *f=fi*, where

fi < fi+1

A* search, evaluation

- Completeness: YES
- Time complexity:
  - Number of nodes expanded is still exponential in the length of the solution.

EFFECTS

Quality of a given heuristic

- Determined by the effective branching factor ***b****
- A *b** close to 1 is ideal
- *N = 1 + b* + (b*)2 + . . . + (b*)d*

N = # nodes

d = solution depth

EXAMPLE

- If A* finds a solution depth 5 using 52 nodes, then b* = 1.91

- Usual *b** exhibited by a given heuristic is fairly constant over a large range of problem instances

. . .

- A well-designed heuristic should have a b* close to 1.

- … allowing fairly large problems to be solved

NUMERICAL EXAMPLE

Comparison of the search costs and effective branching factors for the IDS and A* with h1,& h2.Data are averaged over 100 instances of the 8-puzzle, for various solution lengths

	Search Cost			Effective Branching Factor		
d	IDS	A*(h_1)	A*(h_2)	IDS	A*(h_1)	A*(h_2)
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	364404	227	73	2.78	1.42	1.24
14	3473941	539	113	2.83	1.44	1.23
16	–	1301	211	–	1.45	1.25
18	–	3056	363	–	1.46	1.26
20	–	7276	676	–	1.47	1.27
22	–	18094	1219	–	1.48	1.28
24	–	39135	1641	–	1.48	1.26

Fig. 4.8 Comparison of the search costs and effective branching factors for the IDA and A* algorithms with h1, h2. Data are averaged over 100 instances of the 8-puzzle, for various solution lengths.

INVENTING HEURISTIC FUNCTIONS

- How ?

- Depends on the restrictions of a given problem

- A problem with lesser restrictions is known as a **relaxed problem**

INVENTING HEURISTIC FUNCTIONS

One problem

… one often fails to get one “clearly best” heuristic

Given *h1, h2, h3, … , hm* ; none dominates any others.

Which one to choose ?

$$h(n) = max(h1(n), \ldots, hm(n))$$

INVENTING HEURISTIC FUNCTIONS

Another way:

- performing experiment randomly on a particular problem
- gather results
- decide base on the collected information

Using heuristics in Games

Two agent Games

Minimax Procedure on Exhaustively Search Graphs

- Games have always been an important application area for heuristic algorithm

- Two person games are more complicated than simple puzzle because of the existence of a “hostile” and unpredictable opponent

Minimax Procedure on Exhaustively Search Graphs

- In playing games whose state space may be exhaustively delineated, the primary difficulty is in accounting for the actions of the opponent

- A simple way to handle this assumes that your opponent uses the same knowledge

- Minimax searches the games space under this assumption

Minimax Procedure on Exhaustive Search Graphs

- The opponents in a game are referred to as MIN and MAX
  - MAX represents the player trying to win
  - MIN is the opponent who attempts to minimize MAX's score

Minimax Procedure on Exhaustively Search Graphs

- We label each level in the search space according to whose move it is at the point in the game
- Each leaf node is given a value of 1 or 0, depending on whether a win for MAX (1) or for MIN (0)

Minimax Procedure on Exhaustively Search Graphs

- Minimax propagates these values **up** the graph through successive parent nodes according to the rule:
  - If the parent is a MAX node, give it the maximum value among its children
  - If the parent is a MIN node, give it the minimum value among its children

Minimax Procedure on Exhaustively Search Graphs

- Because all of MIN's possible first moves lead to nodes with a derived values of 1, the second player always can force the game to a win, regardless of MIN's first move

Minimax to Fixed Ply Depth

- In applying minimax to more complicated games, it is seldom possible to expand the state space search graph out to the leaf node
- Instead, the state space is searched to a predefined number of levels
- This is called an *n-ply look ahead*

Minimax to Fixed Ply Depth

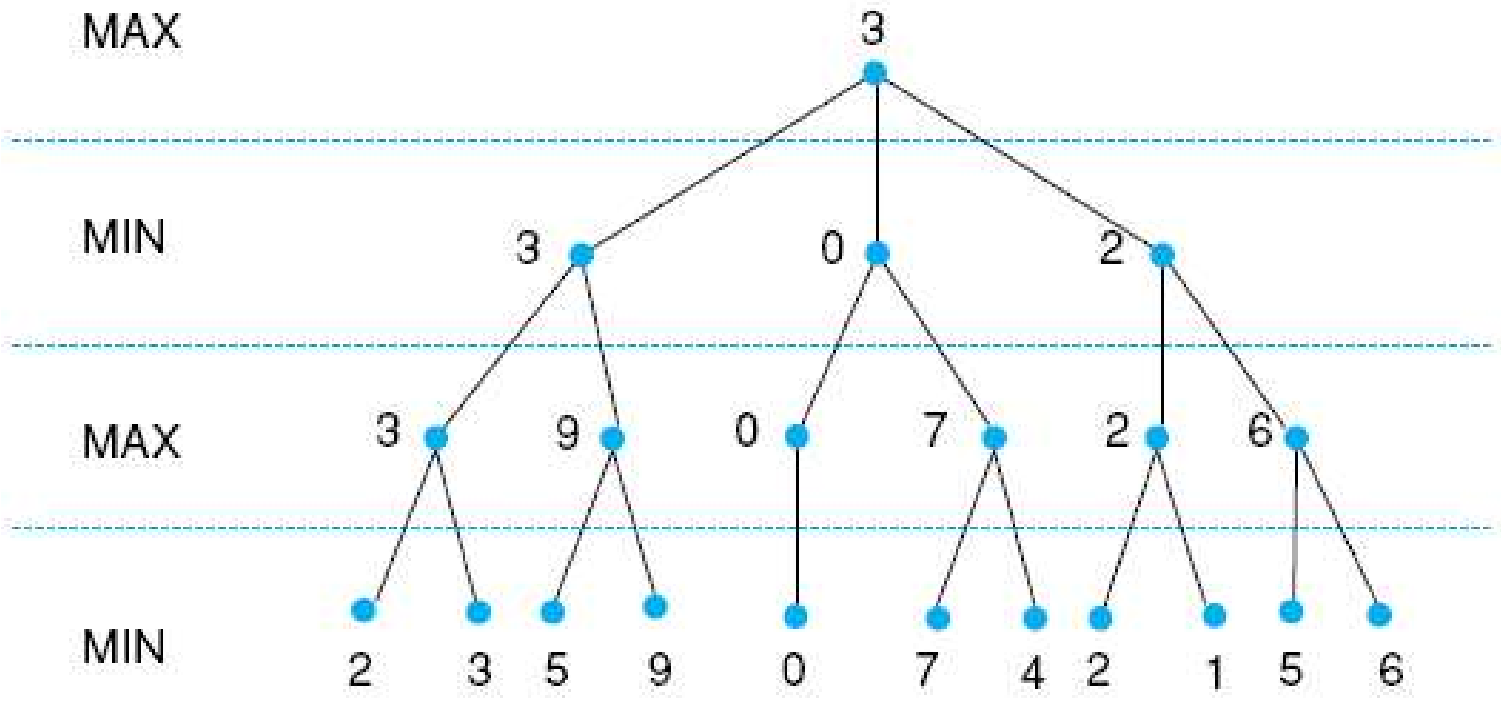
- As the leaves of this subgraph is not complete, it is not possible to give them win or loss

- Instead, each node is given a value according to some heuristic evaluation
  - In chess, you may consider the number of pieces belonging to MAX and MIN
  - A more sophisticated strategy might assign different values to each piece

Minimax to Fixed Ply Depth

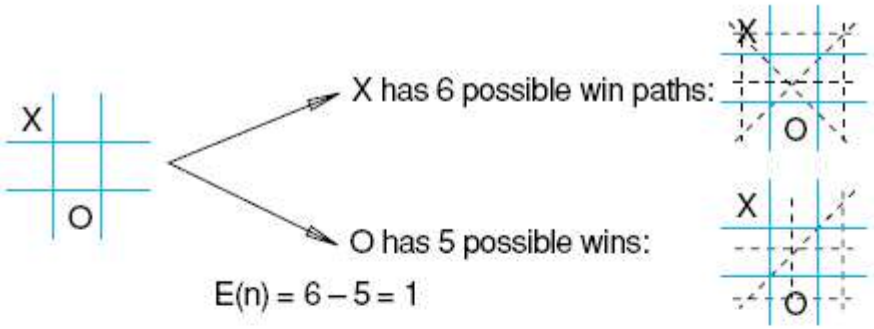
- Minimax propagates these values **up** the graph through successive parent nodes according to the rule:
  - If the parent is a MAX node, give it the maximum value among its children
  - If the parent is a MIN node, give it the minimum value among its children

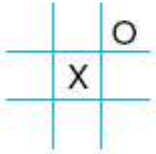
Minimax to Fixed Ply Depth



Tic-Tac-Toe as an example

Fig 4.22 Heuristic measuring conflict applied to states of tic-tac-toe.





Heuristic is E(n) = M(n) – O(n)
where M(n) is the total of My possible winning lines
O(n) is total of Opponent's possible winning lines
E(n) is the total Evaluation for state n

Fig 4.23 Two-ply minimax applied to the opening move of tic-tac-toe, from Nilsson (1971).

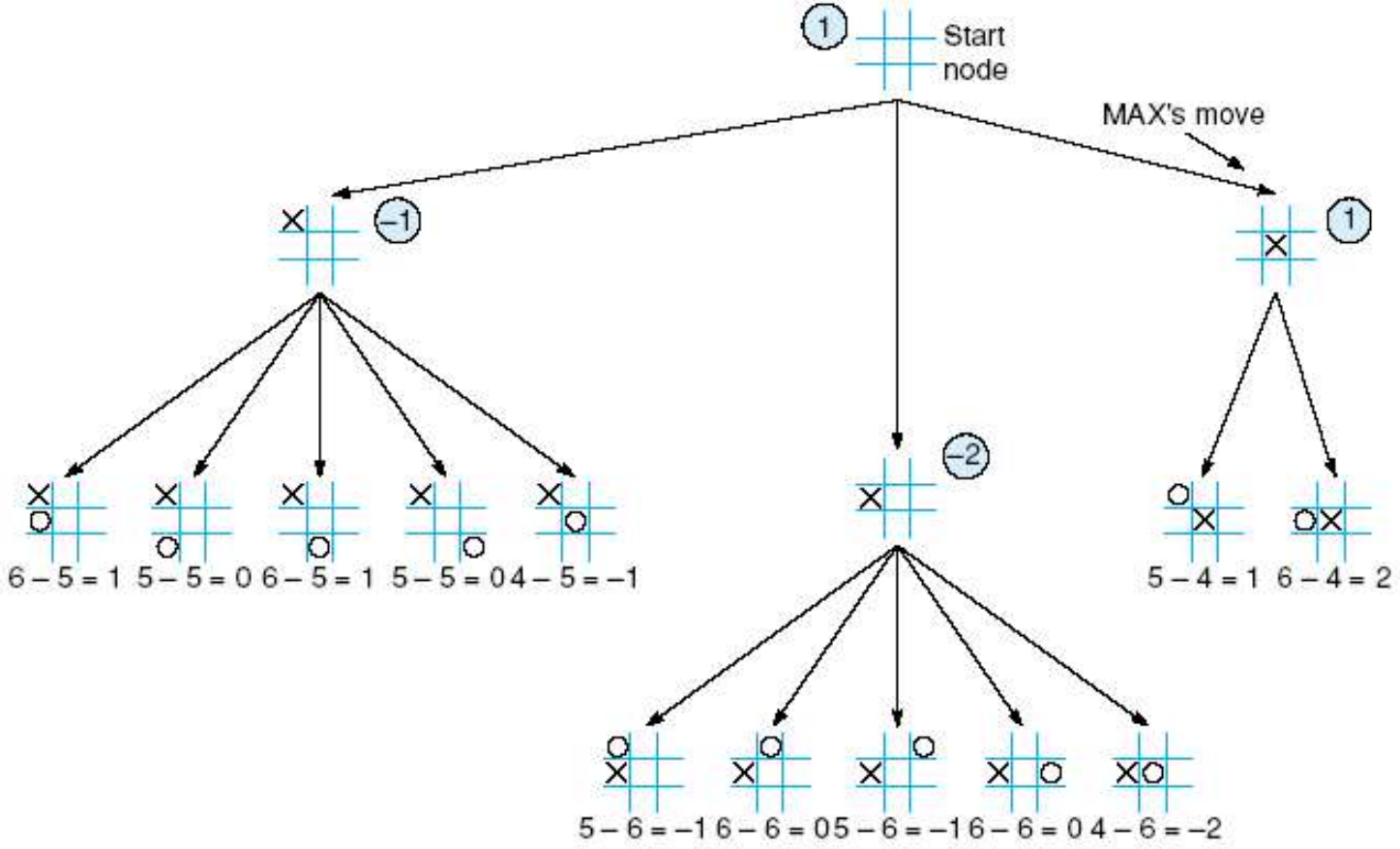


Fig 4.24 Two ply minimax, and one of two possible MAX second moves, from Nilsson (1971).

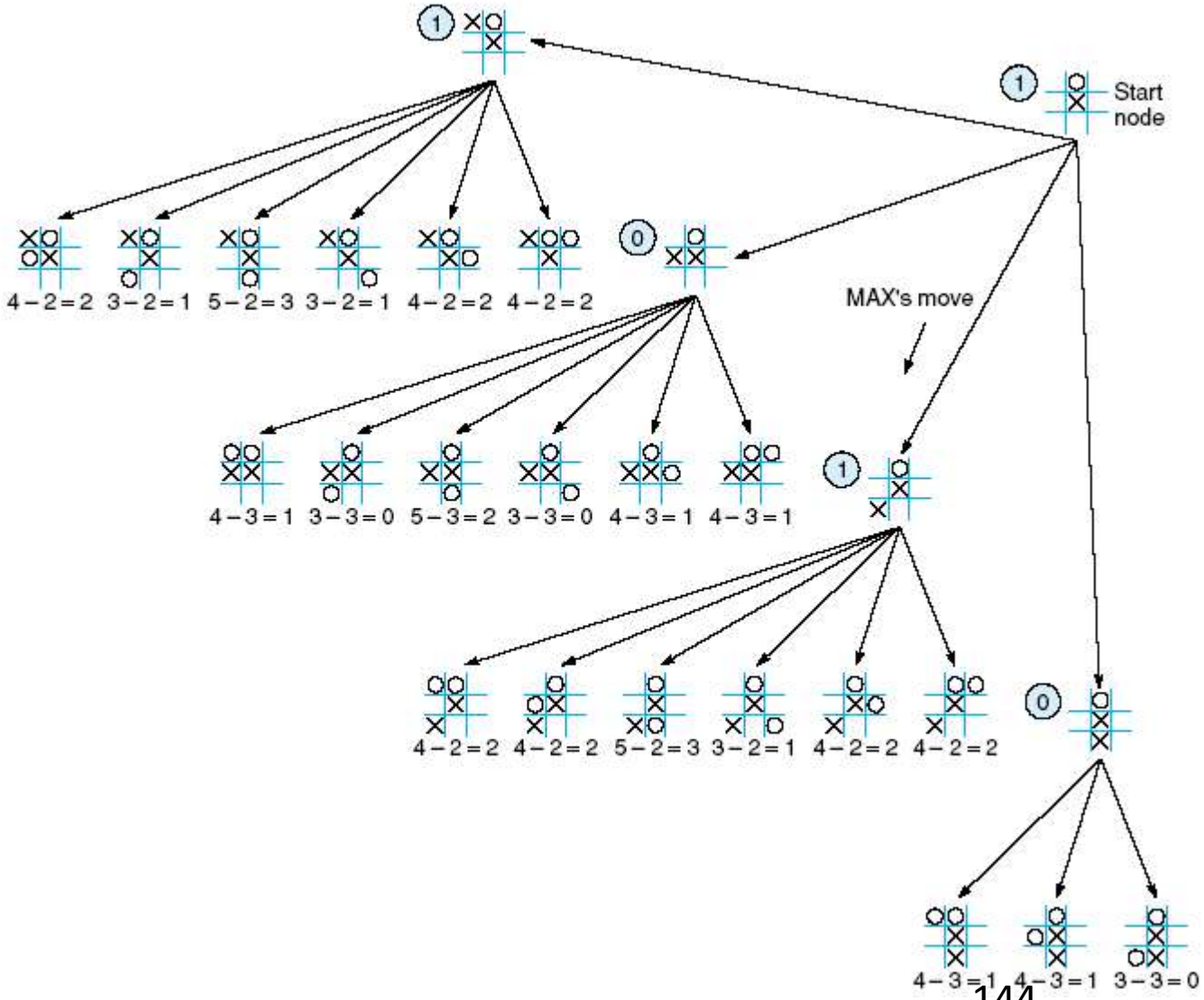
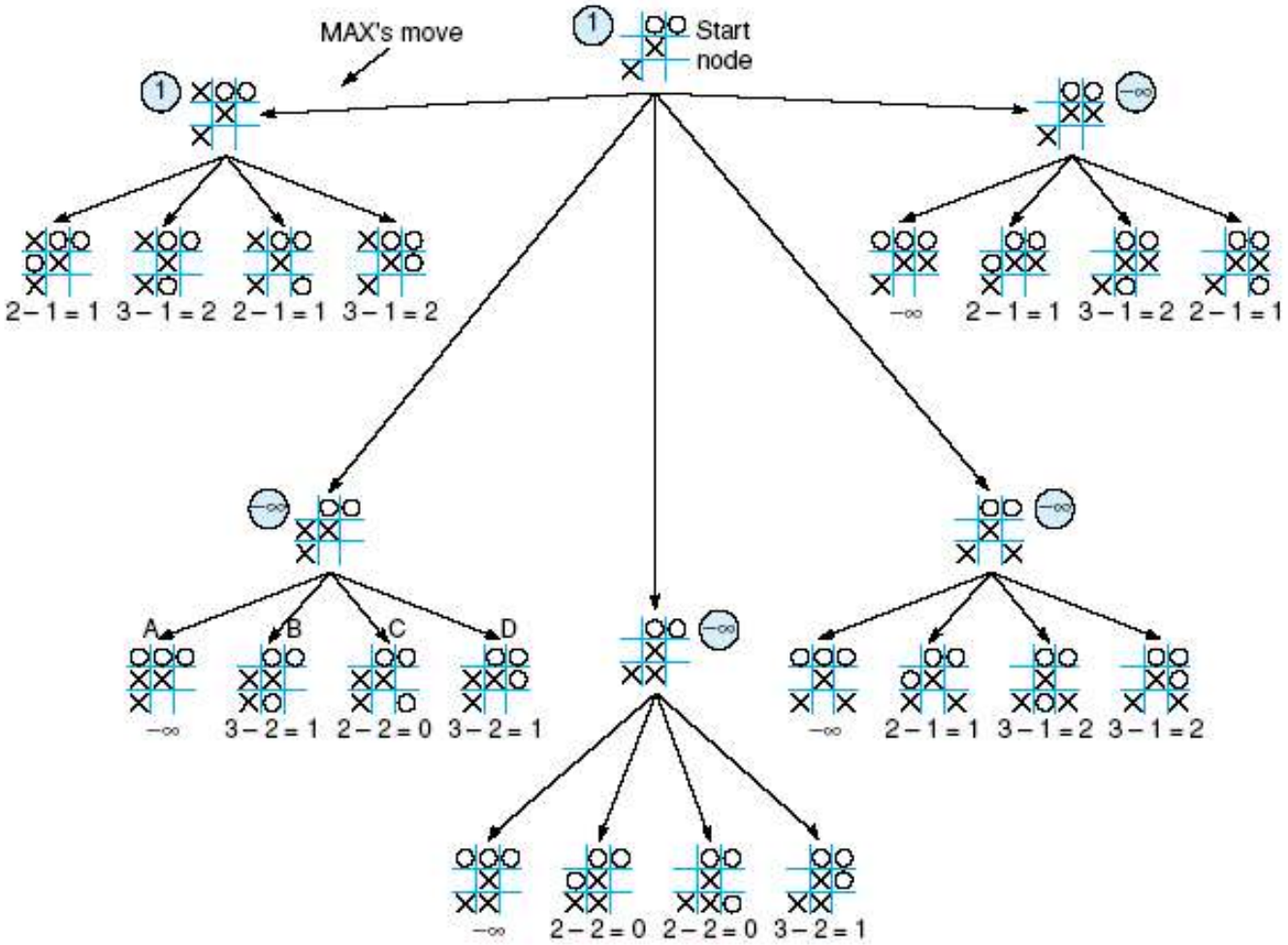


Fig 4.25 Two-ply minimax applied to X's move near the end of the game, from Nilsson (1971).



Game Tree (2-player, Deterministic, Turns)

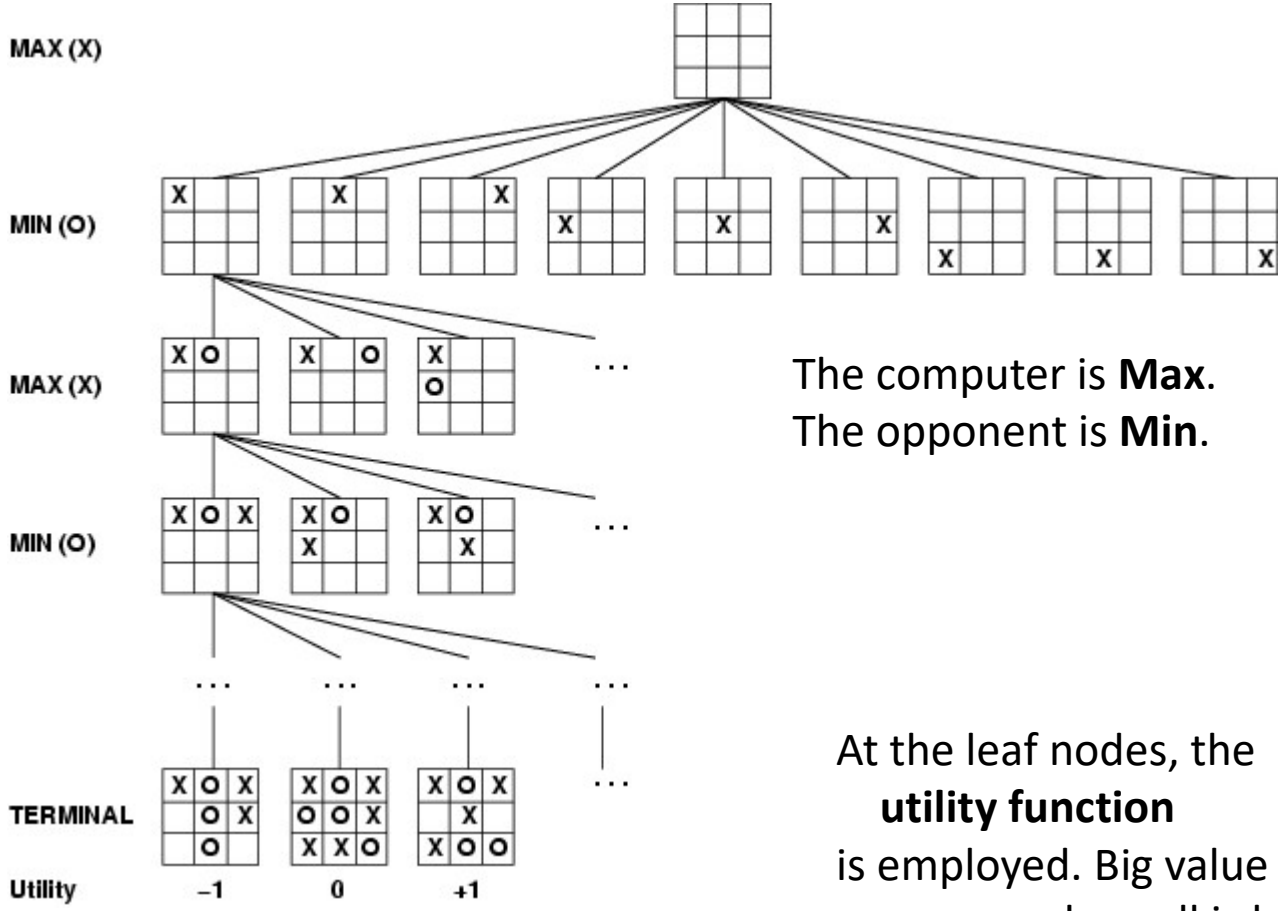
computer's turn

opponent's turn

computer's turn

opponent's turn

leaf nodes are evaluated



Since X is max
If X wins utility value is +1
else if X loses then utility value is -1
else if draw then utility value is 0

The computer is **Max**.
The opponent is **Min**.

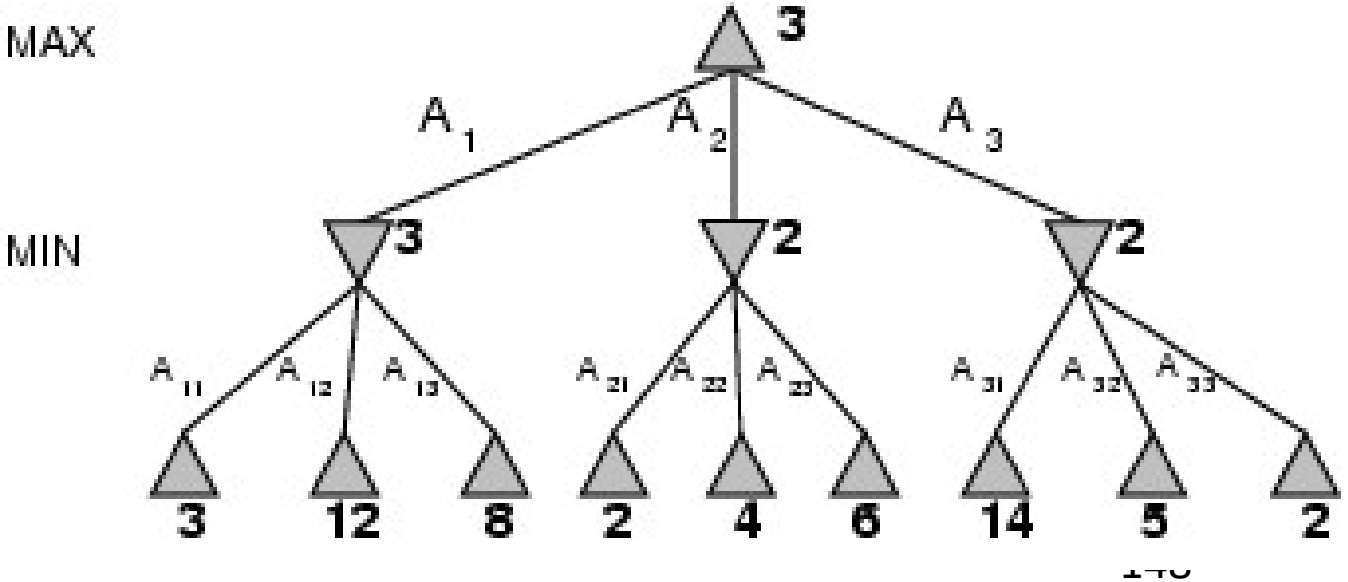
At the leaf nodes, the **utility function** is employed. Big value means good, small is bad.

Mini-Max Terminology

- utility function: the function applied to leaf nodes
- backed-up value
  - of a max-position: the value of its largest successor
  - of a min-position: the value of its smallest successor
- minimax procedure: search down several levels; at the bottom level apply the utility function, back-up values all the way up to the root node, and that node selects the move.

Minimax

- Perfect play for deterministic games
- Idea: choose move to position with highest minimax value
  = best achievable payoff against best play
- E.g., 2-ply game:



Minimax Strategy

- Why do we take the min value every other level of the tree?
- These nodes represent the opponent's choice of move.
- The computer assumes that the human will choose that move that is of least value to the computer.

Terminal State -> Leaf node

Minimax algorithm

```
function MINIMAX-DECISION(state) returns an action
    v ← MAX-VALUE(state)
    return the action in SUCCESSORS(state) with value v

function MAX-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s))
    return v

function MIN-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← ∞
    for a, s in SUCCESSORS(state) do
        v ← MIN(v, MAX-VALUE(s))
    return v
```

At each state if the node is max node chose max of the successor else
choose min of successor

Hence if the node is terminal state directly assign the value of utility function as nodes values

If not then chose successor's value that is the maximum

Similarly for minimum

Properties of Minimax

- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? O(bm)
- Space complexity? O(bm) (depth-first exploration)

- For chess, b ≈ 35, m ≈100 for "reasonable" games
  → exact solution completely infeasible

Need to speed it up.

Alpha-Beta Procedure

Minimax pursues all branches in the space, including many that could be ignored or pruned by a more intelligent algorithm.

1. Rather than searching the entire space to the ply depth, alpha-beta search proceeds in a depth-first fashion

2. Two values alpha and beta are created where alpha value associated with max nodes can never decrease whereas the beta value associated with min nodes can never increase.

- The alpha-beta procedure can speed up a depth-first minimax search.
- Alpha: a lower bound on the value that a max node may ultimately be assigned

  $v \geq \alpha$

  this is the minimum value that a max node can take

- Beta: an upper bound on the value that a minimizing node may ultimately be assigned

  $v \leq \beta$

  this is the maximum value that a min node can take.

So what happens is below Max node we have min node and these min nodes values are what will be propagated to the max node right

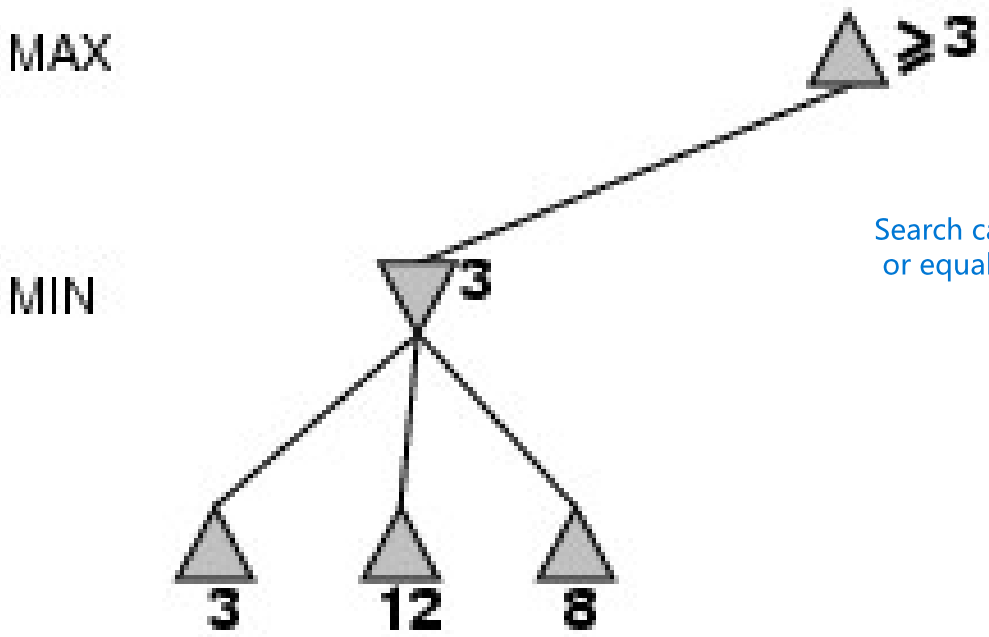
So if the alpha value of max node is 6, then this node need not consider any min nodes with backed up value less than 6 bcz alpha for that max node can never go below 6.

similarly if min node has beta as 6 then need not consider any backed up value which is >= 6 because beta value for this min node can never exceed 6.

α-β pruning example

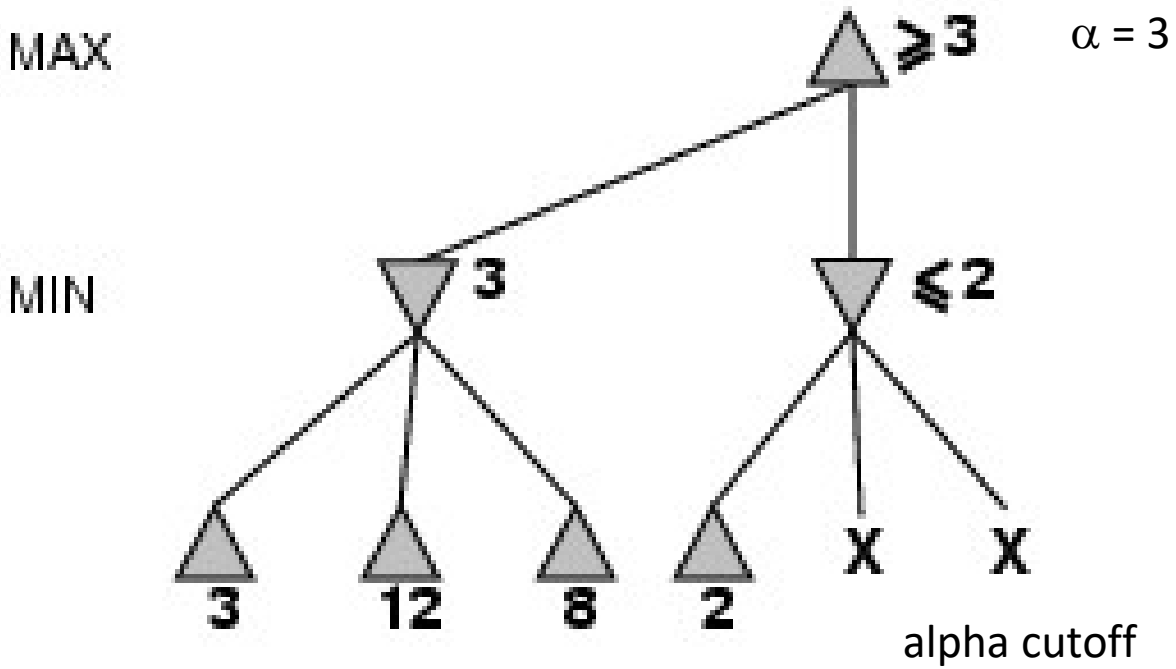
Based on alpha-beta values there are 2 ways in which we can prune the tree:

Search can be stopped below any MIN node having a beta value less than or eq to the alpha value of any of its MAX node ancestors



Search can be stopped below any MAX node having an alpha value greater than or equal to the beta value of any of its MIN node ancestors.

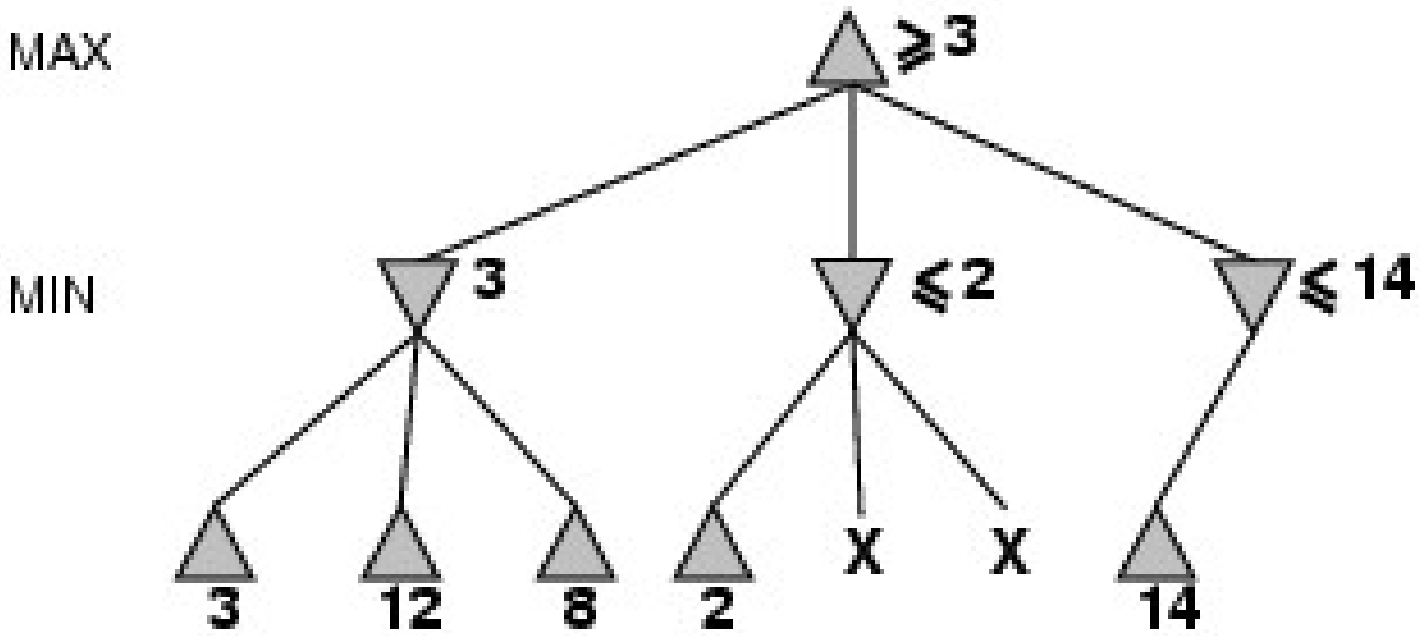
α-β pruning example



here alpha of the node is 3 and the child node has a value 2 then need not consider it because it is less than 3

this is called alpha cutoff.

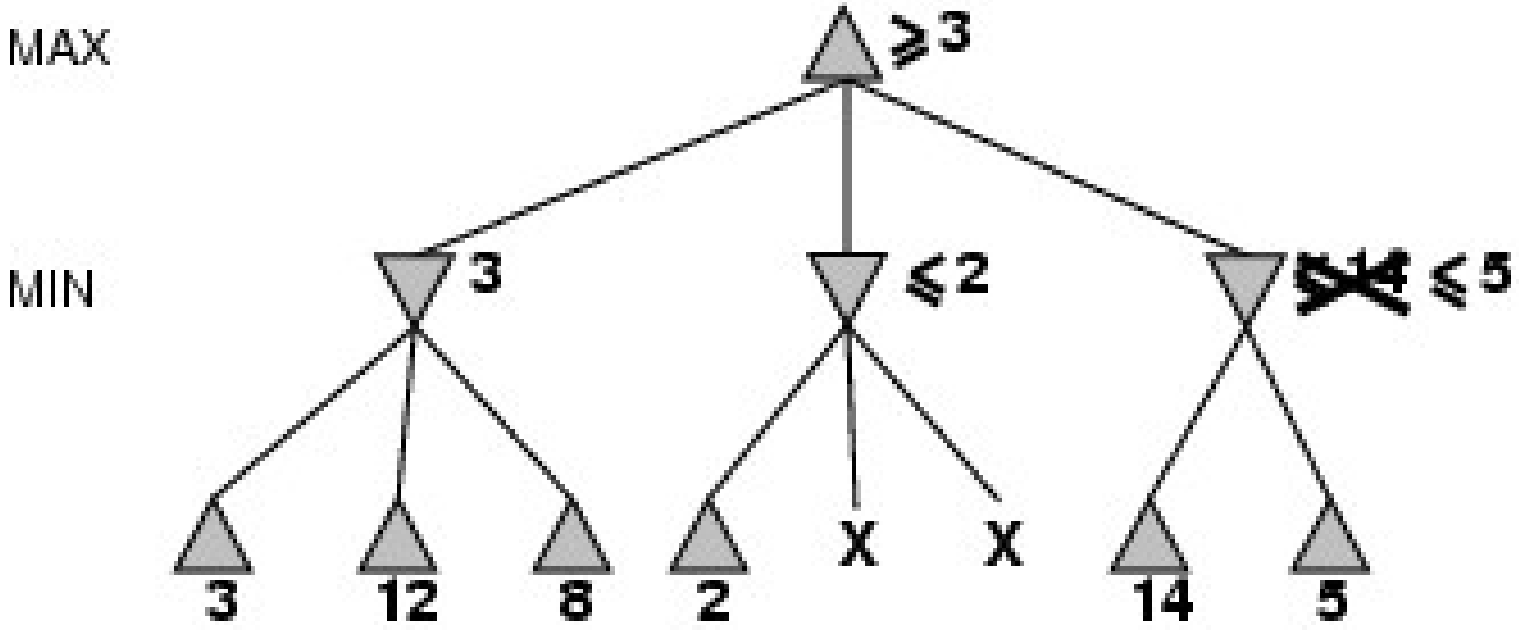
α-β pruning example



we have to consider 14 because 14 is greater than 3 and we can have the node's alpha value increased.

Alpha-beta pruning thus expresses a relatio between nodes at ply n and nodes at ply n + 2 under which entire subtrees rooted a level n + 1 can be eliminated from consideration

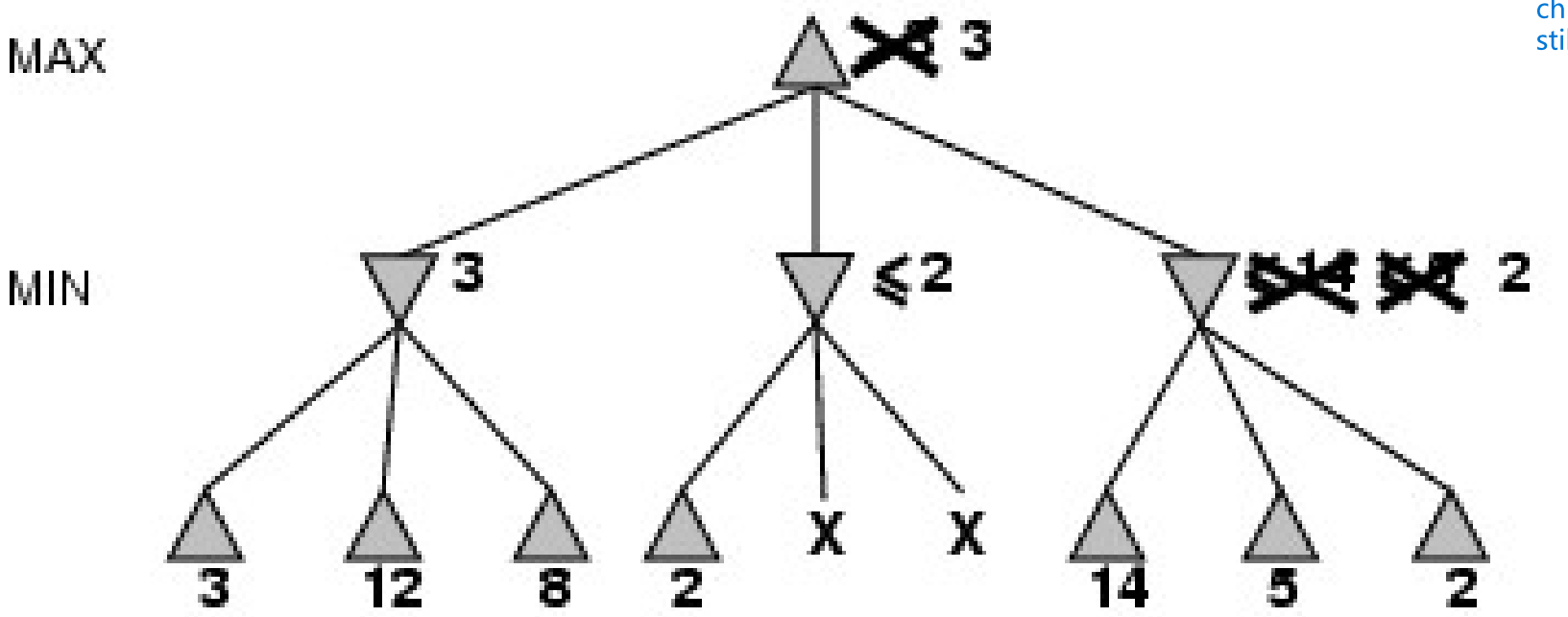
α-β pruning example



Here the min node has seen a max node with alpha value which is 5 which is less than 14 and beta value of a min node can decrease so 14 is updated to 5

now we see that we got 5 that is the max node has alpha 3 and child node has beta 5 so we update the parent's alpha value to 5

α-β pruning example



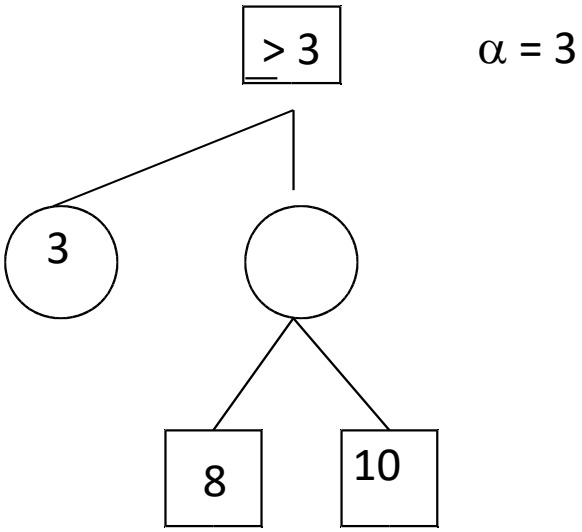
now we see that min node has child with alpha 2 i.e < 3 so we'll still update

so 5 changes to 2

now we have our parent node value which is 5 this should be updated as well which is the max of the child node's beta which is 3

- **The alpha value of a MAX node is set equal to the current largest final backed-up value of its successors**

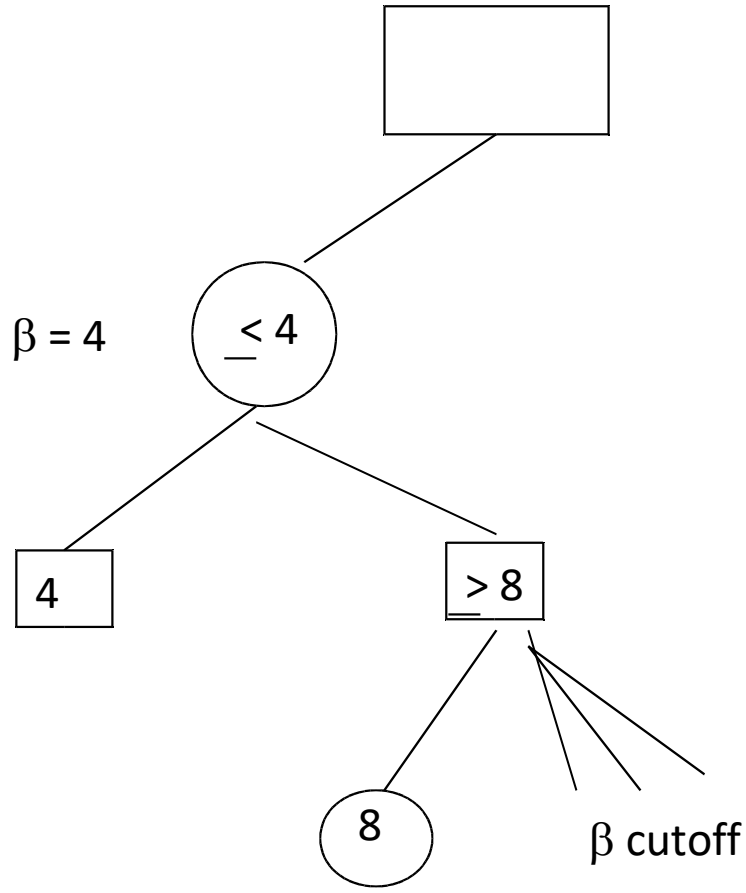
Alpha Cutoff



What happens here? Is there an alpha cutoff?

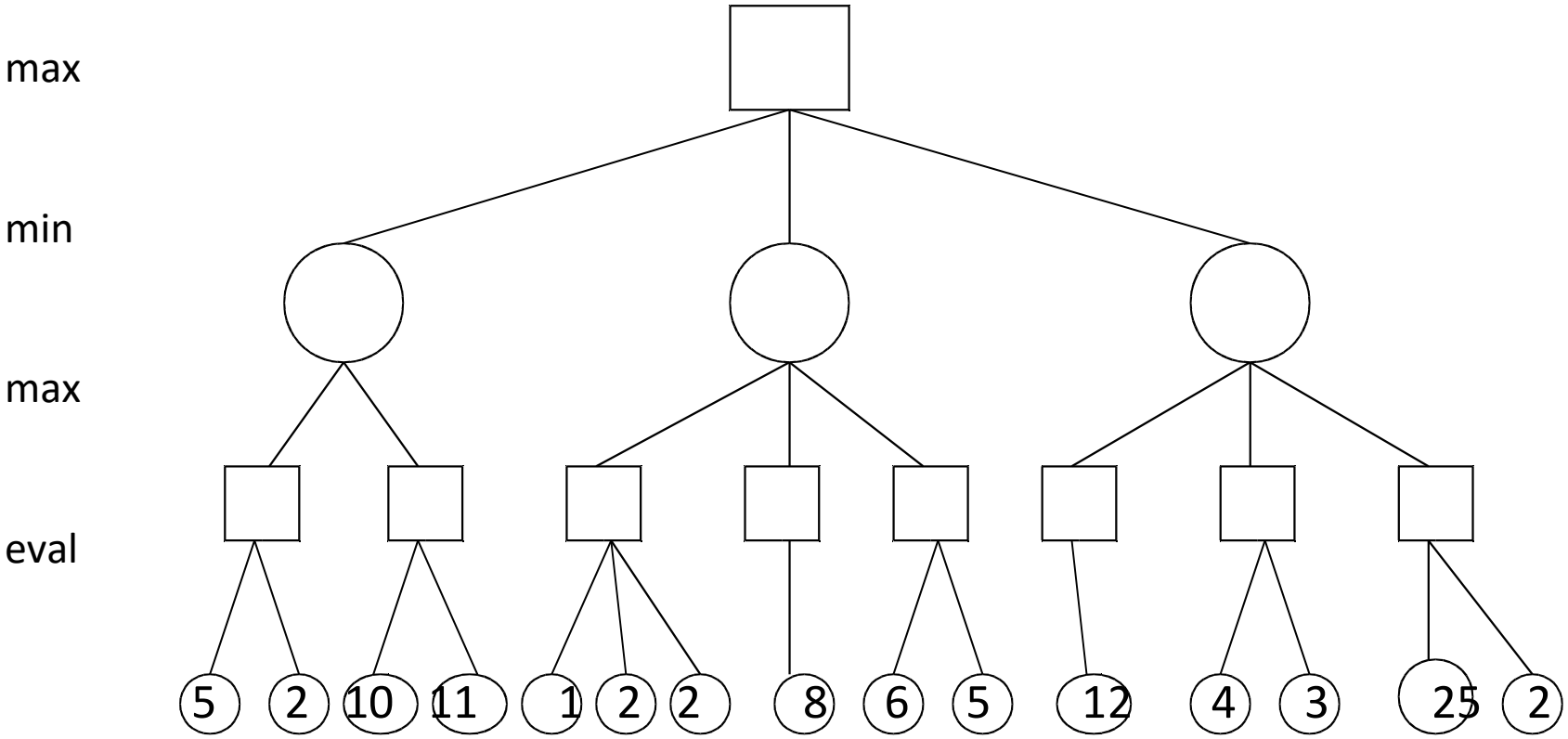
Search can be discontinued under any MIN node having a beta value less than or equal to the alpha value of any of its MAX node ancestors

Beta Cutoff



Search can be discontinued under any MAX node having an alpha value greater than or equal to the beta value of any of its MIN node ancestors

Alpha-Beta Pruning



Properties of α-β

- Pruning does not affect final result. This means that it gets the exact same result as does full minimax.
- Good move ordering improves effectiveness of pruning
- With "perfect ordering," time complexity = O(bm/2)

→ doubles depth of search

The α-β algorithm

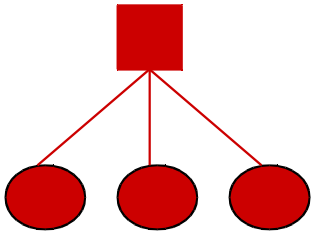
function ALPHA-BETA-SEARCH(*state*) **returns** *an action*
  inputs: *state*, current state in game

  $v \leftarrow$ MAX-VALUE($state, -\infty, +\infty$)
  return the *action* in SUCCESSORS(*state*) with value v

---

function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  inputs: *state*, current state in game
    α, the value of the best alternative for MAX along the path to *state*
    β, the value of the best alternative for MIN along the path to *state*

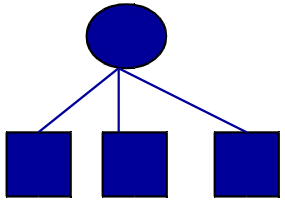
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  $v \leftarrow -\infty$
  for a, s in SUCCESSORS(*state*) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(s, α, β))
    if $v \geq \beta$ **then return** v cutoff
    $\alpha \leftarrow$ MAX(α, v)
  return v



The α-β algorithm

```
function MIN-VALUE(state, α, β) returns a utility value
  inputs: state, current state in game
          α, the value of the best alternative for MAX along the path to state
          β, the value of the best alternative for MIN along the path to state
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for a, s in SUCCESSORS(state) do
    v ← MIN(v, MAX-VALUE(s, α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

cutoff



When do we get alpha cutoffs?

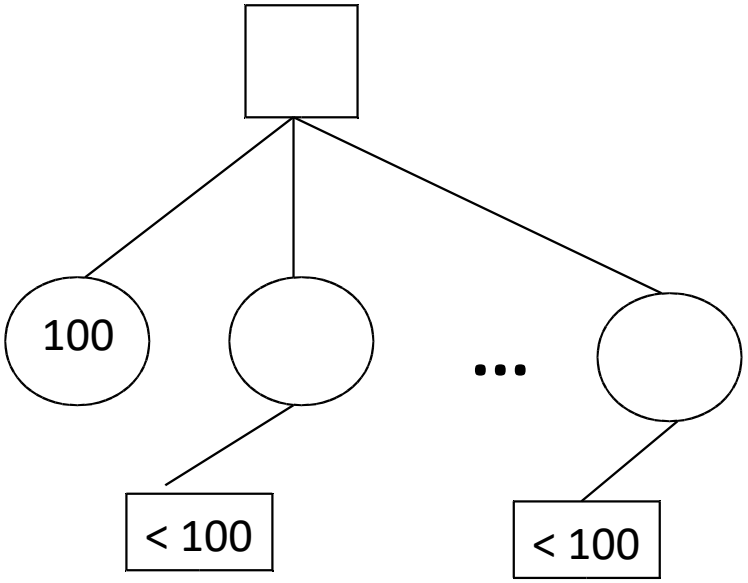
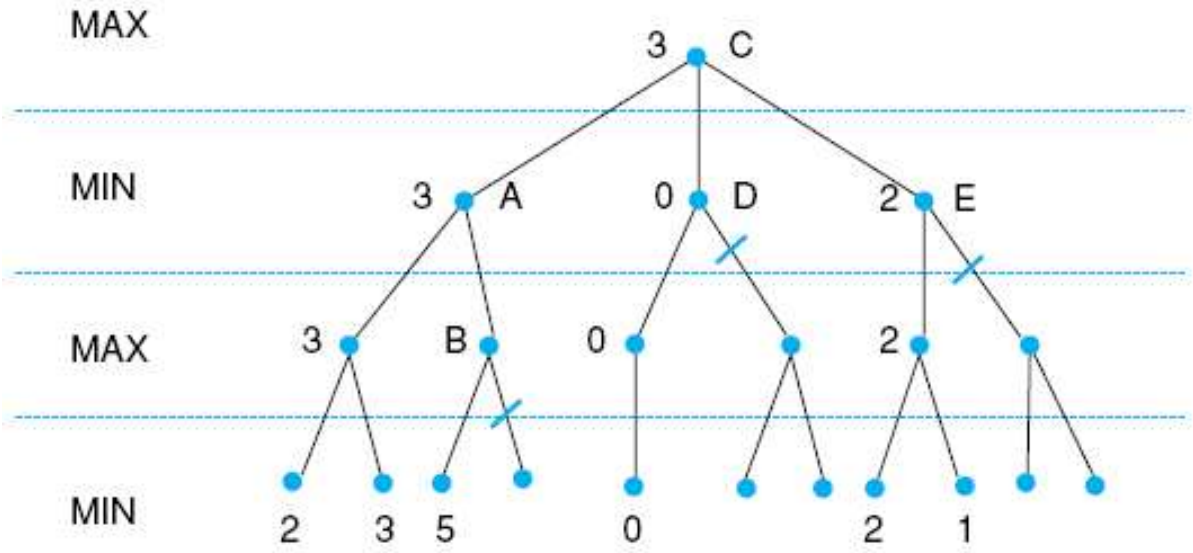


Fig 4.26 Alpha-beta pruning applied to state space of Fig 4.21. States without numbers are not evaluated.



A has $\beta = 3$ (A will be no larger than 3)
B is β pruned, since $5 > 3$
C has $\alpha = 3$ (C will be no smaller than 3)
D is α pruned, since $0 < 3$
E is α pruned, since $2 < 3$
C is 3